Foundations of C Programming (Structured Programming) - Formatted I/O

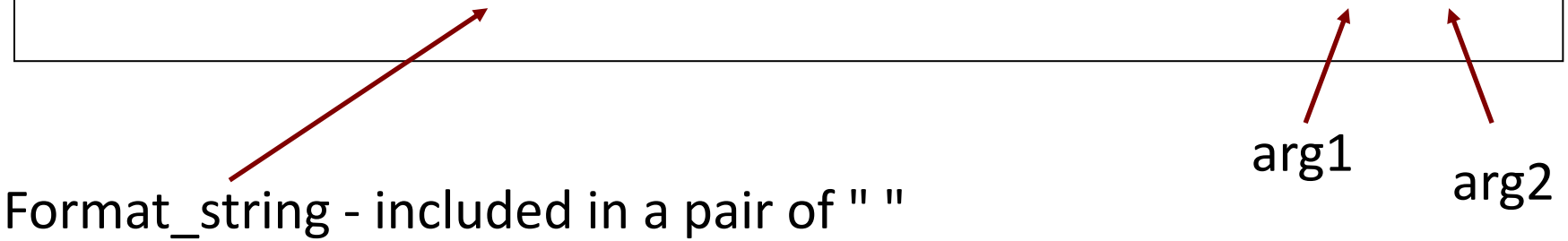
Outline

- Write output to screen - printf
- Read input from keyboard - scanf

printf

- Format: `printf(format_string, arg1, arg2, …,argn);`
- Display the output

```
printf("The results are %d and %d\n", a, b);
```



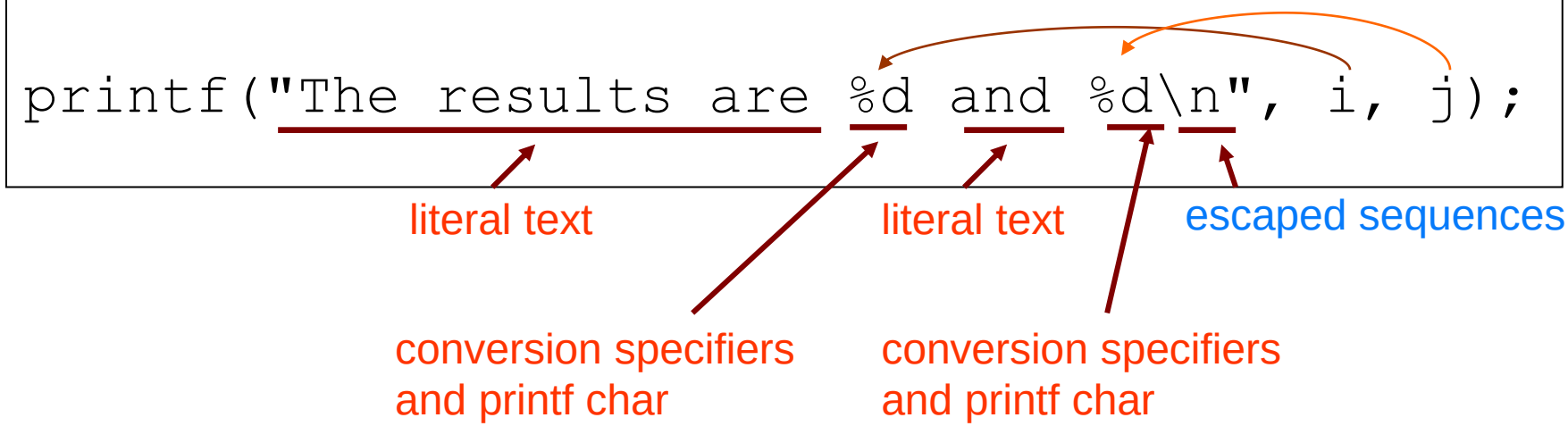
printf

- Format string contains (optional):
  - literal text: printed without any variation
  - escape sequences: preceeded by \, used to print special characters
  - conversion specifiers : % followed by a single character (printf char)
    - Conversion specifier indicates (usually) that the value of a variable is to be displayed at this location. The variables to be printed must appear as arg1, arg2, ….in the order that they appear in the format string.

printf

```
int i = 10, j = 20;

printf("The results are %d and %d\n", i, j);
```



The output is as follows:

```
The results are 10 and 20
```

Conversion Specifiers

Conversion Specifier	Meaning
%c	Single character
%d	Signed decimal integer
%x	Hexadecimal number
%f	Decimal floating point number
%e	Floating point in "scientific notation"
%s	Character string (more on this later)
%u	Unsigned decimal integer
%%	Just print a % sign
%ld	long int

*More conversion specifiers are available. Refer to the Internet and reference books.

Escape Sequences

- Sequence — Meaning
  - \a — Bell (alert)
  - \b — Backspace
  - \n — Newline
  - \t — Horizontal tab
  - \\ — Backslash
  - \' — Single quote
  - \" — Double quotation
  - \x*hh* — ASCII char specified by hex digits *hh*
  - *0oo* — ASCII char specified by octal digits *oo*
  - *1oo* — ASCII char specified by octal digits 1*oo*

Class Exercises

- What is the output of the following program

```
#include <stdio.h>
int main(){
      char c, d;
      float f;
      c = 'd';
      d = 97;
      f = 23.5;
      printf("c = %c, d = %c \nf = %f, f = %e\n", c, d, f, f);
      printf("d = %d \tf = %d", d, f);
      return 0;
}
```

Attention: In printf statement, %d should not match a float number and %f should not match an integer number.

Class Exercises

- What is the output of the following program

```
#include <stdio.h>
int main() {
  int ten = 10, x = 42;
  char ch1 = 'o', ch2 = 'f';
  printf("%d%% %c%c %d is %f\n", ten, ch1, ch2, x,
     1.0 * x/ten );
  return 0;
}
```

scanf

- Format: `scanf(format_string, arg1, arg2, …, argn);`
- Read an input
- Format string is similar to that in printf.

```
int value;
float ratio;

scanf("value=%d,ratio=%f", &value, &ratio);
```

Format_string - included in a pair of ""

arg1

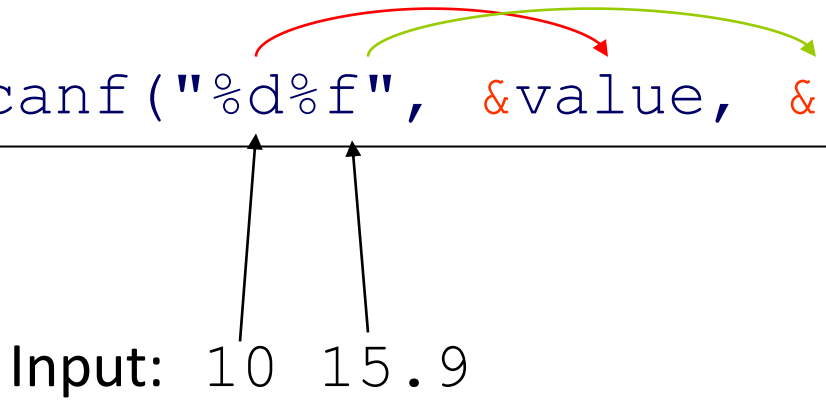
arg2

&: means "address of". Will be explained in the later chapter.

scanf

```
int value;
float ratio;

scanf("%d%f", &value, &ratio);
```



Tip: It is smart not to include any literal text in a *scanf* statement.

%c and space (空格）

- Space in the input means reading next data. It is skipped if the next data is not of char type.

Input from keyboard: 10 20.1	
`scanf("%d%d", &a, &b);`	a = 10, b = 20
`scanf("%d%c%d", &a, &c, &b);`	a = 10, c = ' ', b = 20
`scanf("%d%d%c", &a, &b, &c);`	a = 10, b = 20, c = '.'
`scanf("%d%f%c", &a, &f, &c);`	a = 10, f = 20.1, c='\n'

Class Exercises

- What is the output of the following program (Percentage.c)

```
#include <stdio.h>
int main() {
   int percentage, x;
   char ch1, ch2;
   scanf("%d%c%c%d", &percentage, &ch1, &ch2, &x);
   printf("%d%% %c%c %d is %f\n", percentage, ch1, ch2,
      x, 1.0* x * percentage/100);
   return 0;
}
```

- What is the output if the input is 25of60
- What is the output if the input is 25 o f 60

Attention: *printf* statement should be in one line except that the line is automatically broken in a code editor. You should not break the line by yourself.

Summary

- Make the output in a specific format
- Data can be inputted from keyboard in a sequence.